1. NumPy

1.1 What is NumPy?

NumPy (Numerical Python) is a Python library used for numerical and mathematical operations.

It provides an efficient data structure called an **array**, along with functions for statistics, mathematical calculations, reshaping, filtering, and random-number generation.

A simple way to remember it:

> **NumPy = Numerical foundation for Data Science and Machine Learning**

Pandas also uses NumPy internally for many numerical operations.

1.2 Import NumPy

```
import numpy as np
```

`np` is the commonly used alias for NumPy.

1.3 `np.array()`

What does it do?

Creates a NumPy array from a Python list or another sequence.

Why is it useful?

Machine Learning and numerical computation frequently work with arrays/matrices.

Example

```
import numpy as np

marks = np.array([65, 72, 81, 90, 55])

print(marks)
```

Output:

```
[65 72 81 90 55]
```

2D array

```
data = np.array([
    [65, 72, 81],
    [70, 85, 90],
    [55, 60, 68]
])

print(data)
```

Think of a 2D NumPy array as:

```
Rows × Columns
    3 × 3
```

1.4 shape

What does it do?

Returns the dimensions of an array.

Why is it useful?

It tells us how many rows and columns our data contains.

Example

```
data = np.array([
    [65, 72, 81],
    [70, 85, 90],
    [55, 60, 68]
])

print(data.shape)
```

Output:

```
(3, 3)
```

Meaning:

```
3 rows
3 columns
```

1.5 ndim

What does it do?

Returns the number of dimensions of an array.

Example

```
print(data.ndim)
```

Output:

```
2
```

A one-dimensional array has:

```
ndim = 1
```

A table-like array has:

```
ndim = 2
```

1.6 size

What does it do?

Returns the total number of elements.

Example

```
print(data.size)
```

Output:

```
9
```

Because:

```
3 × 3 = 9
```

1.7 Array Indexing

What does it do?

Allows us to access individual elements.

Example

```
marks = np.array([65, 72, 81, 90, 55])

print(marks[0])
print(marks[2])
```

Output:

```
65
81
```

Remember:

> Python indexing starts from `0`.

1.8 2D Array Indexing

```
data = np.array([
    [65, 72, 81],
    [70, 85, 90],
    [55, 60, 68]
])

print(data[0, 1])
```

Output:

```
72
```

The syntax is:

```
array[row, column]
```

1.9 Slicing

What does it do?

Selects a range of elements.

Example

```
marks = np.array([65, 72, 81, 90, 55])

print(marks[1:4])
```

Output:

```
[72 81 90]
```

Selecting a column from a 2D array

```
print(data[:, 1])
```

This means:

```
:  → all rows
1  → column 1
```

1.10 Arithmetic Operations

What does it do?

Performs mathematical operations on all elements of an array.

Example

```
marks = np.array([50, 60, 70, 80])

print(marks + 5)
print(marks * 2)
```

Output:

```
[55 65 75 85]
[100 120 140 160]
```

This is called **vectorized operation**.

Instead of writing a loop manually, NumPy performs the operation on the complete array.

1.11 `np.sum()`

What does it do?

Calculates the total.

```
marks = np.array([50, 60, 70, 80])

print(np.sum(marks))
```

Output:

```
260
```

Use

Useful for totals such as:

- Total sales
- Total revenue
- Total quantities
- Total scores

1.12 `np.mean()`

What does it do?

Calculates the average.

```
marks = np.array([50, 60, 70, 80])

print(np.mean(marks))
```

Output:

```
65.0
```

1.13 `np.median()`

What does it do?

Finds the middle value after sorting the data.

```
values = np.array([10, 20, 30, 40, 100])

print(np.median(values))
```

Output:

```
30.0
```

Why is it useful?

Median is often useful when data contains **outliers**.

1.14 `np.min()` and `np.max()`

```
values = np.array([10, 20, 30, 40, 100])

print(np.min(values))
print(np.max(values))
```

Output:

```
10
100
```

Useful for understanding the range of a variable.

1.15 `np.std()`

What does it do?

Calculates standard deviation.

```
values = np.array([10, 20, 30, 40, 50])

print(np.std(values))
```

Why is it useful?

Standard deviation tells us how spread out values are around the mean.

This becomes important later when learning:

- Data distribution
- Scaling
- Standardization
- Variance
- Model preprocessing

1.16 `np.arange()`

What does it do?

Creates evenly spaced values using a step.

```
numbers = np.arange(1, 11)

print(numbers)
```

Output:

```
[1 2 3 4 5 6 7 8 9 10]
```

Example with step:

```
np.arange(0, 20, 2)
```

Output:

```
[0 2 4 6 8 10 12 14 16 18]
```

1.17 np.linspace()

What does it do?

Creates a specified number of evenly spaced values between two limits.

```
values = np.linspace(0, 1, 5)

print(values)
```

Output:

```
[0.   0.25 0.5  0.75 1.  ]
```

Useful when we need evenly distributed numerical points, especially for mathematical or visualization tasks.

1.18 reshape()

What does it do?

Changes the shape of an array without changing its data.

```
numbers = np.array([1, 2, 3, 4, 5, 6])

new_data = numbers.reshape(2, 3)

print(new_data)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

This is important because Machine Learning data is frequently represented as a matrix:

```
Rows = Samples
Columns = Features
```

1.19 Boolean Filtering

What does it do?

Selects values based on a condition.

```
marks = np.array([45, 67, 82, 39, 91])

print(marks[marks >= 60])
```

Output:

```
[67 82 91]
```

This introduces an important Data Science concept:

> Select data based on a condition.

1.20 Random Numbers

```
np.random.randint(1, 101, 10)
```

Generates 10 random integers between 1 and 100.

Random numbers are useful for:

- Testing
- Simulations
- Creating sample data
- Machine Learning experiments

NumPy — Core Operations to Remember

Operation	Main Use
`np.array()`	Create an array
`.shape`	Get dimensions

Operation	Main Use
`.ndim`	Get number of dimensions
`.size`	Get total elements
Indexing	Access elements
Slicing	Select ranges
Arithmetic	Perform numerical calculations
`np.sum()`	Total
`np.mean()`	Average
`np.median()`	Middle value
`np.min()`	Minimum
`np.max()`	Maximum
`np.std()`	Standard deviation
`np.arange()`	Generate values with a step
`np.linspace()`	Generate evenly spaced values
`.reshape()`	Change array shape
Boolean filtering	Select values by condition
`np.random`	Generate random values

---

2. Pandas

2.1 What is Pandas?

Pandas is a Python library designed for working with structured and tabular data.

It is one of the most important libraries for Data Science.

A simple way to remember it:

> **Pandas = Read, clean, transform, analyze, and prepare data**

2.2 Import Pandas

```
import pandas as pd
```

2.3 Series

What is a Series?

A Series is a one-dimensional Pandas data structure.

Think of it as a **single column**.

```
ages = pd.Series([21, 25, 30, 28])

print(ages)
```

2.4 DataFrame

What is a DataFrame?

A DataFrame is a two-dimensional table containing rows and columns.

This is the structure you will use most often.

```
data = {
    "Name": ["Amit", "Rahul", "Priya", "Neha"],
    "Age": [21, 25, 30, 28],
    "Marks": [75, 82, 91, 68]
}

df = pd.DataFrame(data)

print(df)
```

Output:

```
    Name  Age  Marks
0   Amit   21     75
1  Rahul   25     82
2  Priya   30     91
3   Neha   28     68
```

2.5 `pd.read_csv()`

What does it do?

Reads a CSV file into a DataFrame.

```
df = pd.read_csv("students.csv")
```

This is usually one of the first operations in a real ML project.

Workflow:

```
CSV File
   ↓
pd.read_csv()
   ↓
DataFrame
   ↓
Analysis / Cleaning
```

2.6 `head()`

What does it do?

Displays the first rows.

```
df.head()
```

By default it displays the first 5 rows.

You can specify the number:

```
df.head(10)
```

Why?

To quickly understand what the dataset looks like.

2.7 `tail()`

Displays the last rows.

```
df.tail()
```

Useful for checking the end of the dataset.

2.8 shape

```
df.shape
```

Example output:

```
(1000, 8)
```

Meaning:

```
1000 rows
8 columns
```

This should be one of the first checks after loading a dataset.

2.9 columns

```
df.columns
```

Shows the column names.

Useful for understanding available features and the target variable.

2.10 dtypes

```
df.dtypes
```

Shows the data type of each column.

For example:

```
Age          int64
Salary     float64
Gender      object
```

This helps identify:

- Numerical columns

- Categorical columns
- Date columns
- Potential data-type problems

2.11 `info()`

```
df.info()
```

Provides:

- Number of rows
- Column names
- Data types
- Non-null counts
- Memory information

This is one of the most useful first checks.

2.12 `describe()`

```
df.describe()
```

Provides statistical information for numerical columns:

- Count
- Mean
- Standard deviation
- Minimum
- 25%
- Median
- 75%
- Maximum

This is extremely useful during EDA.

2.13 Selecting One Column

```
df["Age"]
```

Returns the Age column.

2.14 Selecting Multiple Columns

```
df[["Age", "Marks"]]
```

Returns multiple columns.

Remember the difference:

```
df["Age"]
```

One column.

```
df[["Age", "Marks"]]
```

Multiple columns.

2.15 Selecting Rows with `loc`

`loc` selects rows/columns using labels.

```
df.loc[0]
```

Selects row 0.

You can also select specific columns:

```
df.loc[:, ["Name", "Marks"]]
```

2.16 Selecting Rows with `iloc`

`iloc` uses integer positions.

```
df.iloc[0]
```

First row.

```
df.iloc[0:3]
```

First three rows.

2.17 Filtering Rows

What does it do?

Selects rows based on conditions.

```
df[df["Marks"] > 80]
```

This returns students whose marks are greater than 80.

2.18 Multiple Conditions

```
df[
    (df["Age"] > 25) &
    (df["Marks"] > 70)
]
```

Important operators:

```
&   → AND
|   → OR
~   → NOT
```

Use parentheses around individual conditions.

2.19 `value_counts()`

What does it do?

Counts how frequently each value occurs.

```
df["Gender"].value_counts()
```

Example:

```
Female    520
Male      480
```

Very useful for categorical data.

It can also help inspect class imbalance in classification problems.

2.20 `unique()`

Shows unique values.

```
df["Gender"].unique()
```

Example:

```
["Male", "Female"]
```

2.21 `nunique()`

Counts the number of unique values.

```
df["Gender"].nunique()
```

Useful for understanding categorical columns and identifiers.

2.22 Missing Values — `isnull()`

```
df.isnull()
```

Returns True/False for missing values.

Usually we want the count:

```
df.isnull().sum()
```

Example:

```
Age       5
Salary    12
Gender    0
```

This is an essential operation before Machine Learning.

2.23 `notnull()`

Checks values that are not missing.

```
df.notnull()
```

Useful when filtering valid records.

2.24 `dropna()`

What does it do?

Removes rows/columns containing missing values.

```
df.dropna()
```

Example:

```
clean_df = df.dropna()
```

Important:

> Do not automatically remove all missing-value rows. First understand why data is missing and how much is missing.

2.25 `fillna()`

What does it do?

Replaces missing values.

Example:

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

This replaces missing Age values with the median.

Other possibilities:

```
df["Age"].fillna(25)
```

or:

```
df["City"].fillna("Unknown")
```

Later in ML, missing-value treatment should be performed carefully to avoid data leakage.

2.26 Duplicate Detection

```
df.duplicated()
```

Counts duplicates:

```
df.duplicated().sum()
```

2.27 Remove Duplicates

```
df = df.drop_duplicates()
```

Useful during data cleaning when duplicate records are not valid observations.

2.28 Sorting

```
df.sort_values("Marks")
```

Descending:

```
df.sort_values("Marks", ascending=False)
```

Useful for ranking and exploring data.

2.29 `mean()`, `median()`, `std()`

```
df["Marks"].mean()
df["Marks"].median()
df["Marks"].std()
```

These are useful for understanding numerical variables.

2.30 `min()` and `max()`

```
df["Marks"].min()
df["Marks"].max()
```

Useful for understanding the range.

2.31 `groupby()`

What does it do?

Groups data based on a column and allows calculations for each group.

Example:

```
df.groupby("Gender")["Marks"].mean()
```

Possible output:

```
Gender
Female    78.4
Male      74.8
```

Concept:

```
Gender
  ↓
Split records
  ↓
Calculate average Marks
  ↓
Compare groups
```

2.32 Multiple Aggregations

```
df.groupby("Gender")["Marks"].agg(
    ["mean", "median", "min", "max"]
)
```

Useful for comparing groups.

2.33 `agg()`

Allows multiple aggregation operations.

```
df["Marks"].agg(["mean", "median", "min", "max"])
```

2.34 Creating a New Column

What does it do?

Creates a derived feature.

```
df["Passed"] = df["Marks"] >= 40
```

Now:

```
Marks
  ↓
Condition
  ↓
Passed
```

This is an introduction to **feature engineering**.

2.35 Creating a Numerical Feature

```
df["Bonus_Marks"] = df["Marks"] * 0.10
```

This creates a new feature based on an existing feature.

2.36 `map()`

Useful for transforming individual values.

```
df["Gender_Code"] = df["Gender"].map({
    "Male": 1,
    "Female": 0
})
```

This is a simple example of encoding categorical data.

2.37 `replace()`

Replaces specific values.

```
df["Gender"] = df["Gender"].replace({
    "M": "Male",
    "F": "Female"
})
```

Useful when datasets contain inconsistent labels.

2.38 `astype()`

Changes data type.

```
df["Age"] = df["Age"].astype(int)
```

Useful when a column has an incorrect or unsuitable type.

2.39 `pd.to_datetime()`

Converts a column to datetime.

```
df["Date"] = pd.to_datetime(df["Date"])
```

This is important when working with:

- Sales data
- Transactions
- Trips
- Time series
- Logs

2.40 Working with Dates

After conversion:

```
df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
df["Day"] = df["Date"].dt.day
df["DayOfWeek"] = df["Date"].dt.dayofweek
```

This is another example of feature engineering.

2.41 `drop()`

Removes columns or rows.

Remove a column:

```
df = df.drop("Age", axis=1)
```

Remove multiple columns:

```
df = df.drop(["Age", "Gender"], axis=1)
```

2.42 `rename()`

Changes column names.

```
df = df.rename(columns={
    "Marks": "Final_Marks"
```

```
})
```

Useful for creating clear and consistent feature names.

2.43 concat()

Combines DataFrames vertically or horizontally.

```
combined = pd.concat([df1, df2])
```

Useful when multiple datasets have the same structure.

2.44 merge()

Combines DataFrames using related columns.

```
merged = pd.merge(
    students,
    departments,
    on="Department_ID"
)
```

Concept:

```
Student Data
     +
Department Data
     ↓
   merge()
     ↓
Combined Data
```

This is useful when information is spread across multiple tables.

2.45 corr()

What does it do?

Calculates correlation between numerical columns.

```
df.corr(numeric_only=True)
```

Correlation helps investigate relationships between numerical variables.

Later it becomes useful when studying:

- Feature relationships
- Multicollinearity
- Feature selection

2.46 `to_csv()`

Saves a DataFrame as CSV.

```
df.to_csv("cleaned_data.csv", index=False)
```

2.47 `to_excel()`

Saves a DataFrame as Excel.

```
df.to_excel("cleaned_data.xlsx", index=False)
```

Pandas — Core Operations to Remember

Operation	Main Use
`pd.read_csv()`	Load CSV
`pd.read_excel()`	Load Excel
`head()`	View first rows
`tail()`	View last rows
`shape`	Rows and columns
`columns`	Column names
`dtypes`	Data types
`info()`	Dataset structure
`describe()`	Statistics
`loc`	Label-based selection

Operation	Main Use
`iloc`	Position-based selection
Column selection	Select features
Filtering	Select records by condition
`value_counts()`	Count categories
`unique()`	Find unique values
`nunique()`	Count unique values
`isnull()`	Find missing values
`dropna()`	Remove missing records
`fillna()`	Replace missing values
`duplicated()`	Find duplicates
`drop_duplicates()`	Remove duplicates
`sort_values()`	Sort data
`groupby()`	Group and analyze
`agg()`	Multiple aggregations
`map()`	Transform values
`replace()`	Replace values
`astype()`	Change data type
`to_datetime()`	Convert dates
`.dt`	Extract date components
`drop()`	Remove rows/columns
`rename()`	Rename columns
`concat()`	Combine DataFrames
`merge()`	Join related DataFrames
`corr()`	Calculate correlation
`to_csv()`	Save CSV
`to_excel()`	Save Excel

3. Matplotlib

3.1 What is Matplotlib?

Matplotlib is a Python visualization library.

It allows us to convert numerical and categorical data into graphs so we can visually understand patterns.

A simple way to remember it:

> **Matplotlib = See the data instead of only reading the numbers**

Visualization helps identify:

- Trends
- Relationships
- Distributions
- Skewness
- Outliers
- Differences between groups
- Patterns

3.2 Import Matplotlib

```
import matplotlib.pyplot as plt
```

`pyplot` is the module commonly used for creating plots.

3.3 Basic Plot Structure

A typical Matplotlib workflow is:

```
plt.plot(x, y)

plt.title("My Plot")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")

plt.show()
```

The important idea is:

```
Data
 ↓
Choose chart
 ↓
Add labels/title
```

```
 ↓
Display
```

3.4 Line Plot — `plt.plot()`

What does it do?

Creates a line chart.

Example

```
days = [1, 2, 3, 4, 5]
sales = [100, 120, 150, 140, 180]

plt.plot(days, sales)

plt.title("Daily Sales")
plt.xlabel("Day")
plt.ylabel("Sales")

plt.show()
```

When to use?

Useful for:

- Trends
- Time-based data
- Continuous changes

Example question:

> "How did sales change over time?"

3.5 Scatter Plot — `plt.scatter()`

What does it do?

Shows the relationship between two numerical variables.

```
age = [20, 22, 25, 28, 30, 35]
salary = [25000, 28000, 32000, 40000, 45000, 60000]

plt.scatter(age, salary)
```

```
plt.title("Age vs Salary")
plt.xlabel("Age")
plt.ylabel("Salary")

plt.show()
```

Why is it important for ML?

It helps investigate relationships between features and can reveal:

- Positive relationships
- Negative relationships
- Clusters
- Outliers
- Non-linear patterns

3.6 Histogram — `plt.hist()`

What does it do?

Shows the distribution of a numerical variable.

```
marks = [45, 50, 55, 60, 65, 65, 70, 72, 75, 80, 85, 90, 95]

plt.hist(marks)

plt.title("Marks Distribution")
plt.xlabel("Marks")
plt.ylabel("Frequency")

plt.show()
```

What can we learn?

A histogram can help identify:

- Center
- Spread
- Distribution shape
- Skewness
- Possible unusual values

This is one of the most important visualizations in EDA.

3.7 Boxplot — `plt.boxplot()`

What does it do?

Shows the distribution of numerical data using:

- Median
- Quartiles
- Spread
- Potential outliers

```
salary = [25000, 27000, 30000, 32000, 35000, 38000, 40000, 150000]

plt.boxplot(salary)

plt.title("Salary Distribution")
plt.ylabel("Salary")

plt.show()
```

The value `150000` may appear as an outlier.

Why is it useful for ML?

Boxplots are very useful when investigating **outliers** before model training.

3.8 Bar Chart — `plt.bar()`

What does it do?

Compares values across categories.

```
departments = ["IT", "HR", "Sales", "Finance"]
employees = [50, 20, 45, 30]

plt.bar(departments, employees)

plt.title("Employees by Department")
plt.xlabel("Department")
plt.ylabel("Employees")

plt.show()
```

Use bar charts when the main variable is categorical.

3.9 Horizontal Bar Chart — `plt.barh()`

```
plt.barh(departments, employees)

plt.title("Employees by Department")
plt.xlabel("Employees")
plt.ylabel("Department")

plt.show()
```

Useful when category names are long.

3.10 Pie Chart — `plt.pie()`

```
sizes = [50, 30, 20]
labels = ["Product A", "Product B", "Product C"]

plt.pie(sizes, labels=labels)

plt.title("Product Distribution")

plt.show()
```

Use sparingly.

For Data Science/ML analysis, bar charts are often easier to compare than pie charts.

3.11 Labels

```
plt.xlabel("Age")
plt.ylabel("Salary")
```

Why?

Labels explain what each axis represents.

A graph without proper labels is difficult to interpret.

3.12 Title

```
plt.title("Age vs Salary")
```

The title tells the viewer what the graph represents.

3.13 `plt.legend()`

Useful when multiple datasets or lines are shown.

```
plt.plot(days, sales_a, label="Product A")
plt.plot(days, sales_b, label="Product B")

plt.legend()

plt.show()
```

3.14 Grid

```
plt.grid()
```

A grid can make values easier to read.

3.15 Figure Size

```
plt.figure(figsize=(8, 5))
```

Useful when the default graph is too small.

Example:

```
plt.figure(figsize=(8, 5))

plt.hist(marks)

plt.title("Marks Distribution")
plt.xlabel("Marks")
plt.ylabel("Frequency")
```

```
plt.show()
```

3.16 Multiple Plots

Matplotlib allows several plots to be created in one figure.

```
plt.subplot(1, 2, 1)

plt.hist(marks)
plt.title("Marks Distribution")

plt.subplot(1, 2, 2)

plt.boxplot(marks)
plt.title("Marks Boxplot")

plt.show()
```

Concept:

```
        Figure
    ┌─────────────────┐
    │ Histogram       │ Boxplot
    │                 │
    └─────────────────┘
```

This is useful when comparing different views of the same variable.

3.17 Saving a Plot

```
plt.savefig("marks_distribution.png")
```

Useful for:

- Reports
- Presentations
- Documentation
- Project results

Matplotlib — Core Operations to Remember

Operation	Main Use
`plt.plot()`	Line/trend plot
`plt.scatter()`	Relationship between variables
`plt.hist()`	Distribution
`plt.boxplot()`	Distribution + outliers
`plt.bar()`	Category comparison
`plt.barh()`	Horizontal category comparison
`plt.pie()`	Proportional comparison
`plt.xlabel()`	X-axis label
`plt.ylabel()`	Y-axis label
`plt.title()`	Chart title
`plt.legend()`	Identify multiple series
`plt.grid()`	Improve readability
`plt.figure()`	Control figure size
`plt.subplot()`	Multiple plots
`plt.savefig()`	Save visualization
`plt.show()`	Display visualization

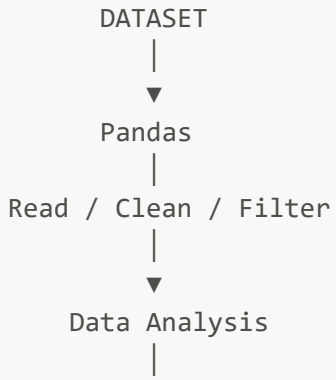
---

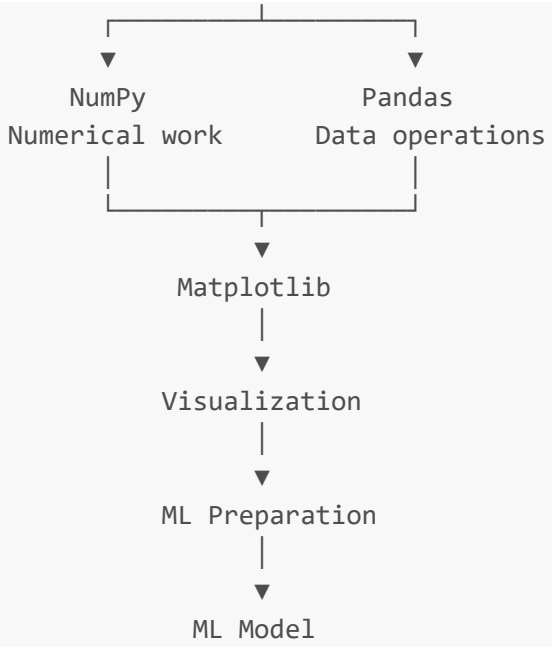
4. How NumPy, Pandas and Matplotlib Work Together

This is the most important concept of the session.

They are not three completely separate tools.

They commonly work together.





5. Complete Mini Example

Let's combine all three libraries.

Step 1 — Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2 — Create Example Data

```
data = {
    "Age": [21, 22, 25, 28, 30, 32, 35, 40],
    "Study_Hours": [2, 3, 4, 5, 6, 7, 8, 9],
    "Marks": [45, 50, 55, 62, 68, 74, 82, 90]
}

df = pd.DataFrame(data)
```

Step 3 — Understand the Data Using Pandas

```
print(df.head())
print(df.shape)
```

```
print(df.info())
print(df.describe())
```

Questions to answer:

- How many rows?
- How many columns?
- What are the data types?
- What is the average Marks?
- What is the minimum Marks?
- What is the maximum Marks?

Step 4 — Use NumPy

```
average_marks = np.mean(df["Marks"])
median_marks = np.median(df["Marks"])
std_marks = np.std(df["Marks"])

print("Average:", average_marks)
print("Median:", median_marks)
print("Standard Deviation:", std_marks)
```

Now we have numerical statistics.

Step 5 — Visualize with Matplotlib

Study Hours vs Marks

```
plt.scatter(df["Study_Hours"], df["Marks"])

plt.title("Study Hours vs Marks")
plt.xlabel("Study Hours")
plt.ylabel("Marks")

plt.show()
```

Now ask:

> Does studying more appear to be associated with higher marks?

Do not automatically claim causation. A graph can show a relationship, not prove that one variable causes another.

Step 6 — Marks Distribution

```
plt.hist(df["Marks"])

plt.title("Marks Distribution")
plt.xlabel("Marks")
plt.ylabel("Frequency")

plt.show()
```

Ask:

> How are the marks distributed?

Step 7 — Marks Boxplot

```
plt.boxplot(df["Marks"])

plt.title("Marks Boxplot")
plt.ylabel("Marks")

plt.show()
```

Ask:

> Are there any possible outliers?

---

6. Practical Exercise

Create a file:

```
student_data.csv
```

with columns:

```
Student_ID
Age
Gender
Study_Hours
Attendance
Assignments
Marks
```

Part A — Pandas

1. Load the dataset

```
pd.read_csv()
```

2. Understand the dataset

Use:

```
head()
shape
columns
dtypes
info()
describe()
```

3. Analyze categorical data

Use:

```
unique()
nunique()
value_counts()
```

4. Check data quality

Find:

```
Missing values
Duplicates
Invalid values
```

5. Filter students

Examples:

```
Students with Marks > 80
Students with Attendance > 75
Students with Study_Hours > 5
```

6. Calculate statistics

Find:

```
Average Marks
Median Marks
Average Study Hours
Highest Marks
Lowest Marks
```

7. Group the data

Calculate:

```
Average Marks by Gender
Average Marks by Attendance group
```

8. Create a feature

For example:

```
df["Passed"] = df["Marks"] >= 40
```

---

7. NumPy Practical Tasks

Using numerical columns from the dataset:

1. Convert a column to a NumPy array.
2. Calculate mean.
3. Calculate median.
4. Calculate standard deviation.
5. Find minimum and maximum.
6. Apply a mathematical operation.
7. Filter values using a condition.
8. Reshape a small NumPy array.
9. Experiment with `arange()`.
10. Experiment with `linspace()`.

---

8. Matplotlib Practical Tasks

Create at least these visualizations:

1. Histogram

```
Marks distribution
```

2. Scatter plot

```
Study Hours vs Marks
```

3. Scatter plot

```
Attendance vs Marks
```

4. Boxplot

```
Marks
```

5. Bar chart

```
Average Marks by Gender
```

6. Line plot

Create a simple trend using suitable numerical data.

For every graph, interns should answer:

> **What does this graph tell us?**

NumPy

> NumPy is mainly used for numerical computation and array-based operations.

Pandas

> Pandas is used to load, inspect, clean, transform, and analyze structured data.

Matplotlib

> Matplotlib is used to visualize data and discover patterns, distributions, relationships, and possible outliers.

They should also understand this workflow:

```
Load Data
    ↓
Pandas
    ↓
Understand Data
    ↓
Clean Data
    ↓
Analyze Data
    ↓
NumPy + Pandas
    ↓
Visualize
    ↓
Matplotlib
    ↓
Find Patterns
    ↓
Prepare Data
    ↓
Machine Learning
```

---

10. Important Operations for the ML Journey

The interns do not need to memorize every operation immediately.

The following operations are especially important because they will repeatedly appear in Machine Learning projects.

NumPy

```
array
shape
ndim
size
indexing
slicing
arithmetic
sum
mean
median
min
max
std
reshape
arange
linspace
```

```
random
boolean filtering
```

Pandas

```
read_csv
read_excel
head
tail
shape
columns
dtypes
info
describe
loc
iloc
filtering
value_counts
unique
nunique
isnull
dropna
fillna
duplicated
drop_duplicates
sort_values
groupby
agg
map
replace
astype
to_datetime
dt
drop
rename
concat
merge
corr
to_csv
to_excel
```

Matplotlib

```
plot
scatter
hist
boxplot
bar
barh
```

```
pie
xlabel
ylabel
title
legend
grid
figure
subplot
savefig
show
```

---

11. Final Mental Model

Remember the three libraries using one simple sentence:

> **NumPy calculates, Pandas manages data, and Matplotlib helps us see the data.**

And in a Machine Learning project:

```
          RAW DATA
             │
             ▼
          PANDAS
     Load + Inspect
             │
             ▼
          PANDAS
   Clean + Transform
             │
             ▼
     NUMPY + PANDAS
    Calculate + Analyze
             │
             ▼
       MATPLOTLIB
     Visualize Data
             │
             ▼
     Understand Data
             │
             ▼
    Machine Learning
```